

Project NIMO



- April 21, 2026

Project NIMO - Interactive Animated Desktop Companion

Meet NIMO - a smart, expressive desktop companion that reacts to motion, shows emotions, displays time, weather, and more! Powered by Xiao ESP32-C3 and MPU6050 gyroscope, NIMO brings life to your desk with animated eyes and multiple moods.



| What is NIMO?

NIMO is an interactive desktop companion with a 1.3" OLED display that shows animated eyes that follow your movements! When you shake NIMO, it gets dizzy and then automatically becomes angry – all through motion sensing. NIMO also displays:

✓ 🕒 Real-time clock with AM/PM indicator

✓ 🌐 World clock (Jaipur, London, New York)

✓ ☀️ Live weather data and 3-day forecast

✓ 😊 Multiple emotions: Happy, Sad, Angry, Sleepy, Love, Surprised, Suspicious, Dizzy

✓ 🖐️ Touch-sensitive controls to change moods and navigate menus

| How NIMO Works

Motion Tracking: The MPU6050 gyroscope detects tilt and shake. When you tilt NIMO, the eyes follow your movement smoothly. When shaken, NIMO enters a dizzy state with spinning eyes and stars, then automatically becomes angry!

Touch Control: The TTP223 touch sensor lets you navigate menus:

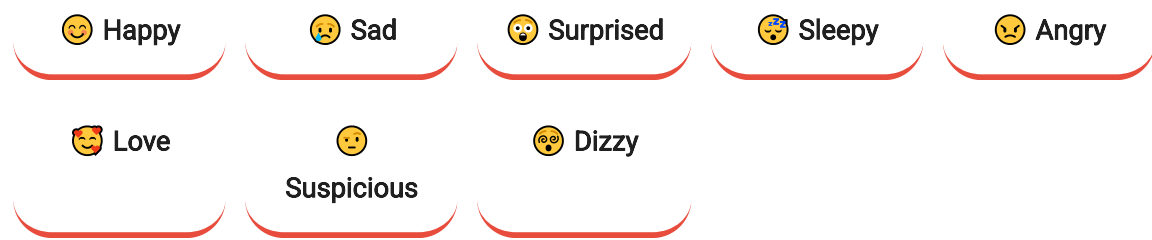
✓ **Tap once:** Switch between screens (Eyes → Clock → Weather)

✓ **Long press:** Change mood or toggle between clock/forecast views

WiFi Features: Connects to WiFi to fetch current weather, 3-day forecast, and sync accurate time via NTP.

Battery Powered: The TP4056 charging module and 1000mAh LiPo battery make NIMO fully portable!

| Emotions & Moods



| Materials Required

Xiao ESP32-C3	-	Buy Here
1.3" OLED Display (SH1106)	-	Buy Here
TP4056 Charging Module	-	Buy Here
3.7V LiPo Battery 1000mAh	-	Buy Here
TTP223 Touch Sensor	-	Buy Here
MPU6050 Gyroscope	-	Buy Here
4mm Slide Switch	-	Buy Here
Jumper Wires	-	Buy Here

| 3D Printable Files

Download the 3D printable enclosure files for NIMO:

Download 3D Files

Print these files using JLC3DP's professional 3D printing service:

[Order 3D Prints from JLC3DP](#)

Pin Connections

Component	Xiao ESP32-C3 Pin	Notes
OLED SDA	GPIO6	I2C Data
OLED SCL	GPIO7	I2C Clock
TTP223 Touch Sensor	GPIO4	Input with pullup
MPU6050 SDA	GPIO6 (shared with OLED)	I2C Data
MPU6050 SCL	GPIO7 (shared with OLED)	I2C Clock
Slide Switch	Battery + to TP4056 IN+	Power ON/OFF
TP4056 OUT+	Xiao 5V Pin	Power input

⚠ Important: The OLED and MPU6050 share the same I2C pins (GPIO6 and GPIO7). This works fine as they have different I2C addresses!

Assembly Steps

1 3D Print the Enclosure

Download and 3D print the enclosure files. NIMO has a cute rounded body with a screen cutout on the front.

2 Install Libraries

Install required libraries in Arduino IDE: Adafruit_GFX, Adafruit_SH110X, MPU6050_tockn, Arduino_JSON, and WiFi/HttpClient (built-in).

3 Upload Code

Change WiFi credentials and OpenWeatherMap API key in the code. Select Xiao ESP32-C3 board, correct COM port, and upload.

4 Wire Components

Solder wires according to the pin table. Connect OLED, MPU6050, touch sensor, battery, and charging module.

5 Mount Inside Enclosure

Secure all components inside the 3D printed enclosure. Ensure the OLED faces the cutout and the touch sensor is accessible.

6 Calibrate & Test

Power on NIMO, keep it still during calibration (first 5 seconds). Then tilt, shake, and tap to test all features!

| System Demo Video



I Built an ESP32 Desk Robot with Motion Sensing, We

Simple Circuits



Watch on

Arduino Code

Project NIMO - Complete Code

Copy Code

```
// =====  
// NIMO - PERFECT EYES & SMALL MOUTH + MPU6050 MOTION  
// Features: FAST 3D eye tracking, Shake = Dizzy → Angry  
// automatically  
// =====  
  
#include <WiFi.h>  
#include <WebServer.h>  
#include <Preferences.h>  
#include <HTTPClient.h>  
#include <Arduino_JSON.h>  
#include <Wire.h>  
#include <Adafruit_GFX.h>  
#include <Adafruit_SH110X.h>  
#include "time.h"  
#include <math.h>  
#include <Fonts/FreeSansBold18pt7b.h>  
#include <Fonts/FreeSansBold9pt7b.h>  
#include <Fonts/FreeSans9pt7b.h>  
  
// MPU6050 Library  
#include <MPU6050_tockn.h>  
MPU6050 mpu6050(Wire);  
  
// =====  
// YOUR CREDENTIALS - EDIT THESE  
// =====  
const char* WIFI_SSID = "Galaxy";
```

```

const char* WIFI_PASSWORD = "rdj02480";
const char* OPENWEATHER_API_KEY = "d9c460c0d5b726695a5553885ac10025";
const char* CITY = "NYC";
const char* COUNTRY_CODE = "USA";
const char* TIMEZONE = "IST-5:30";

// =====
// HARDWARE CONFIGURATION
// =====

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define SDA_PIN 6
#define SCL_PIN 7
#define TOUCH_PIN 4

Adafruit_SH1106G display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

// =====
// MPU6050 MOTION VARIABLES
// =====

#define CALIBRATION_SAMPLES 100
float calibAngleX = 0, calibAngleY = 0, calibAngleZ = 0;
bool isCalibrated = false;

// Current orientation
float currentRoll = 0, currentPitch = 0;

// Shake detection
float shakeIntensity = 0;
bool isShaking = false;
unsigned long shakeStartTime = 0;
unsigned long shakeEndTime = 0;
const int SHAKE_DURATION = 1500;

// Post-dizzy anger (AUTOMATIC)
bool isAngry = false;
unsigned long angryEndTime = 0;
const int ANGRY_DURATION = 2000;

// Fast eye tracking
float targetPupilX = 0, targetPupilY = 0;
float currentPupilX = 0, currentPupilY = 0;
const float SMOOTH_FACTOR = 0.4; // Fast response

// Previous values for shake detection
float lastAccelX = 0, lastAccelY = 0, lastAccelZ = 0;
float prevRoll = 0, prevPitch = 0;

// =====
// WEATHER ICONS
// =====

const unsigned char bmp_clear[] PROGMEM = { 0x00, 0x00, 0x00, 0x00,
0x00, 0x01, 0x80, 0x00, 0x00, 0x01, 0x80, 0x00, 0x00, 0x01, 0x80,
0x00, 0x00, 0x00, 0x00, 0x00, 0x01, 0x03, 0xc0, 0x80, 0x00, 0x0f,
0xf0, 0x00, 0x00, 0x3f, 0xfc, 0x00, 0x00, 0x7f, 0xfe, 0x00, 0x00,

```

```

0xff, 0xff, 0x00, 0x06, 0xff, 0xff, 0x60, 0x06, 0xff, 0xff, 0x60,
0x06, 0xff, 0xff, 0x60, 0x00, 0xff, 0xff, 0x00, 0x3e, 0xff, 0xff,
0x7c, 0x3e, 0xff, 0xff, 0x7c, 0x3e, 0xff, 0xff, 0x7c, 0x00, 0xff,
0xff, 0x00, 0x06, 0xff, 0xff, 0x60, 0x06, 0xff, 0xff, 0x60, 0x06,
0xff, 0xff, 0x60, 0x00, 0xff, 0xff, 0x00, 0x00, 0x7f, 0xfe, 0x00,
0x00, 0x3f, 0xfc, 0x00, 0x01, 0x0f, 0xf0, 0x80, 0x00, 0x03, 0xc0,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x01, 0x80, 0x00, 0x00, 0x01,
0x80, 0x00, 0x00, 0x01, 0x80, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00 };

const unsigned char bmp_clouds[] PROGMEM = { 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x03, 0xe0,
0x00, 0x00, 0x0f, 0xf8, 0x00, 0x00, 0x1f, 0xfc, 0x00, 0x00, 0x3f,
0xfe, 0x00, 0x00, 0x3f, 0xff, 0x00, 0x00, 0x7f, 0xff, 0x80, 0x00,
0xff, 0xff, 0xc0, 0x00, 0xff, 0xff, 0xe0, 0x01, 0xff, 0xff, 0xf0,
0x03, 0xff, 0xff, 0xf8, 0x07, 0xff, 0xff, 0xfc, 0x07, 0xff, 0xff,
0xfc, 0x0f, 0xff, 0xff, 0xfe, 0x0f, 0xff, 0xff, 0xfe, 0x1f, 0xff,
0xff, 0xff, 0x1f, 0xff, 0xff, 0xff, 0x1f, 0xff, 0xff, 0xff, 0x1f,
0xff, 0xff, 0xff, 0x1f, 0xff, 0xff, 0xff, 0x1f, 0xff, 0xff, 0xff,
0x0f, 0xff, 0xff, 0xfe, 0x07, 0xff, 0xff, 0xfc, 0x03, 0xff, 0xff,
0xf8, 0x00, 0xff, 0xff, 0xe0, 0x00, 0x3f, 0xff, 0x80, 0x00, 0x0f,
0xfe, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00 };

const unsigned char bmp_rain[] PROGMEM = { 0x00, 0x00, 0x00, 0x00,
0x00, 0x03, 0xe0, 0x00, 0x00, 0x0f, 0xf8, 0x00, 0x00, 0x1f, 0xfc,
0x00, 0x00, 0x3f, 0xfe, 0x00, 0x00, 0x7f, 0xff, 0x80, 0x00, 0xff,
0xff, 0xc0, 0x01, 0xff, 0xff, 0xf0, 0x03, 0xff, 0xff, 0xf8, 0x07,
0xff, 0xff, 0xfc, 0x0f, 0xff, 0xff, 0xfe, 0x1f, 0xff, 0xff, 0xff,
0x1f, 0xff, 0xff, 0xff, 0x1f, 0xff, 0xff, 0xff, 0x1f, 0xff, 0xff,
0xff, 0x0f, 0xff, 0xff, 0xfe, 0x07, 0xff, 0xff, 0xfc, 0x03, 0xff,
0xff, 0xf8, 0x00, 0xff, 0xff, 0xe0, 0x00, 0x3f, 0xff, 0x80, 0x00,
0x0f, 0xfe, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x60, 0x0c, 0x00,
0x00, 0x60, 0x0c, 0x00, 0x00, 0xe0, 0x1c, 0x00, 0x00, 0xc0, 0x18,
0x00, 0x03, 0x80, 0x70, 0x00, 0x03, 0x80, 0x70, 0x00, 0x03, 0x00,
0x60, 0x00, 0x02, 0x00, 0x40, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00 };

const unsigned char mini_sun[] PROGMEM = { 0x00, 0x00, 0x01, 0x80,
0x00, 0x00, 0x10, 0x08, 0x04, 0x20, 0x03, 0xc0, 0x27, 0xe4, 0x07,
0xe0, 0x07, 0xe0, 0x27, 0xe4, 0x03, 0xc0, 0x04, 0x20, 0x10, 0x08,
0x00, 0x00, 0x01, 0x80, 0x00, 0x00 };

const unsigned char mini_cloud[] PROGMEM = { 0x00, 0x00, 0x00, 0x00,
0x01, 0xc0, 0x07, 0xe0, 0x0f, 0xf0, 0x1f, 0xf8, 0x1f, 0xf8, 0x3f,
0xfc, 0x3f, 0xfc, 0x7f, 0xfe, 0x3f, 0xfe, 0x1f, 0xfc, 0x0f, 0xf0,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00 };

const unsigned char mini_rain[] PROGMEM = { 0x00, 0x00, 0x00, 0x00,
0x01, 0xc0, 0x07, 0xe0, 0x0f, 0xf0, 0x1f, 0xf8, 0x1f, 0xf8, 0x3f,
0xfc, 0x3f, 0xfc, 0x7f, 0xfe, 0x3f, 0xfe, 0x1f, 0xfc, 0x00, 0x00,
0x44, 0x44, 0x22, 0x22, 0x11, 0x11 };

const unsigned char bmp_tiny_drop[] PROGMEM = { 0x10, 0x38, 0x7c,
0xfe, 0xfe, 0x7c, 0x38, 0x00 };

const unsigned char bmp_dizzy_stars[] PROGMEM = { 0x08, 0x20, 0x14,
0x50, 0x22, 0x88, 0x41, 0x04, 0x82, 0x02, 0x41, 0x04, 0x22, 0x88,
0x14, 0x50, 0x08, 0x20, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00 };

const unsigned char bmp_angry_mark[] PROGMEM = { 0x00, 0x00, 0x00,
0x00, 0x08, 0x00, 0x1c, 0x00, 0x3e, 0x00, 0x7f, 0x00, 0x3e, 0x00,

```

```

0x1c, 0x00, 0x08, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00 };

// =====
// EMOTION PARTICLES
// =====
const unsigned char bmp_heart[] PROGMEM = { 0x00, 0x00, 0x0c, 0x60,
0x1e, 0xf0, 0x3f, 0xf8, 0x7f, 0xfc, 0x7f, 0xfc, 0x7f, 0xfc, 0x3f,
0xf8, 0x1f, 0xf0, 0x0f, 0xe0, 0x07, 0xc0, 0x03, 0x80, 0x01, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00 };
const unsigned char bmp_zzz[] PROGMEM = { 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x3c, 0x00, 0x0c, 0x00, 0x18, 0x00, 0x30, 0x00,
0x7e, 0x00, 0x00, 0x3c, 0x00, 0x0c, 0x00, 0x18, 0x00, 0x30, 0x00,
0x7c, 0x00, 0x00, 0x00, 0x00, 0x00 };

// =====
// GLOBALS
// =====
int currentPage = 0;
int subPage = 0;
int tapCounter = 0;
unsigned long lastTapTime = 0;
bool lastPinState = false;
unsigned long pressStartTime = 0;
bool isLongPressHandled = false;
const unsigned long LONG_PRESS_TIME = 800;
const unsigned long DOUBLE_TAP_DELAY = 300;

// MOODS
#define MOOD_NORMAL 0
#define MOOD_HAPPY 1
#define MOOD_SURPRISED 2
#define MOOD_SLEEPY 3
#define MOOD_ANGRY 4
#define MOOD_SAD 5
#define MOOD_EXCITED 6
#define MOOD_LOVE 7
#define MOOD_SUSPICIOUS 8
#define MOOD_DIZZY 9
int currentMood = MOOD_HAPPY;
int weatherMood = MOOD_HAPPY;

// Weather Data
float temperature = 0.0;
float feelsLike = 0.0;
int humidity = 0;
String weatherMain = "Clear";
String weatherDesc = "Sunny";
unsigned long lastWeatherUpdate = 0;

struct ForecastDay {
    String dayName;
    int temp;
    String iconType;
};

```

```

ForecastDay fcast[3];

// =====
// EYE PHYSICS ENGINE
// =====

struct Eye {
    float x, y;
    float w, h;
    float targetX, targetY, targetW, targetH;
    float pupilX, pupilY;
    float targetPupilX, targetPupilY;
    float velX, velY, velW, velH;
    float pVelX, pVelY;
    float k = 0.15;
    float d = 0.65;
    float pk = 0.2;    // FAST pupil response
    float pd = 0.6;
    bool blinking;
    unsigned long lastBlink;
    unsigned long nextBlinkTime;

    void init(float _x, float _y, float _w, float _h) {
        x = targetX = _x;
        y = targetY = _y;
        w = targetW = _w;
        h = targetH = _h;
        pupilX = targetPupilX = 0;
        pupilY = targetPupilY = 0;
        nextBlinkTime = millis() + random(1000, 4000);
    }

    void update() {
        float ax = (targetX - x) * k;
        float ay = (targetY - y) * k;
        float aw = (targetW - w) * k;
        float ah = (targetH - h) * k;
        velX = (velX + ax) * d;
        velY = (velY + ay) * d;
        velW = (velW + aw) * d;
        velH = (velH + ah) * d;
        x += velX;
        y += velY;
        w += velW;
        h += velH;

        float pax = (targetPupilX - pupilX) * pk;
        float pay = (targetPupilY - pupilY) * pk;
        pVelX = (pVelX + pax) * pd;
        pVelY = (pVelY + pay) * pd;
        pupilX += pVelX;
        pupilY += pVelY;
    }
};

Eye leftEye, rightEye;

```

```

unsigned long lastSaccade = 0;
unsigned long saccadeInterval = 3000;
float breathVal = 0;

// =====
// MPU6050 FUNCTIONS
// =====

void calibrateMPU6050() {
    display.clearDisplay();
    display.setFont(NULL);
    display.setCursor(20, 20);
    display.print("Calibrating...");
    display.setCursor(10, 35);
    display.print("Keep device STILL");
    display.display();

    float sumX = 0, sumY = 0, sumZ = 0;

    for(int i = 0; i < CALIBRATION_SAMPLES; i++) {
        mpu6050.update();
        sumX += mpu6050.getAngleX();
        sumY += mpu6050.getAngleY();
        sumZ += mpu6050.getAngleZ();
        delay(10);
    }

    calibAngleX = sumX / CALIBRATION_SAMPLES;
    calibAngleY = sumY / CALIBRATION_SAMPLES;
    calibAngleZ = sumZ / CALIBRATION_SAMPLES;
    isCalibrated = true;

    display.clearDisplay();
    display.setCursor(25, 30);
    display.print("Calibrated!");
    display.display();
    delay(1000);
}

void updateMotion() {
    if (!isCalibrated) return;

    mpu6050.update();

    // Get calibrated angles
    currentRoll = mpu6050.getAngleX() - calibAngleX;
    currentPitch = mpu6050.getAngleY() - calibAngleY;

    // Get acceleration for shake detection
    float accelX = mpu6050.getAccX();
    float accelY = mpu6050.getAccY();
    float accelZ = mpu6050.getAccZ();

    // Calculate shake intensity
    float deltaX = fabs(accelX - lastAccelX);
    float deltaY = fabs(accelY - lastAccelY);

```

```

float deltaZ = fabs(accelZ - lastAccelZ);
shakeIntensity = (deltaX + deltaY + deltaZ) * 10;

// SHAKE DETECTION - Trigger dizzy
if (shakeIntensity > 20.0 && !isShaking && !isAngry && currentPage
== 0) {
    isShaking = true;
    shakeStartTime = millis();
    shakeEndTime = millis() + SHAKE_DURATION;
    currentMood = MOOD_DIZZY;
}

// End dizzy and start ANGRY automatically
if (isShaking && millis() >= shakeEndTime) {
    isShaking = false;
    isAngry = true;
    angryEndTime = millis() + ANGRY_DURATION;
    currentMood = MOOD_ANGRY;
}

// End angry state
if (isAngry && millis() >= angryEndTime) {
    isAngry = false;
    currentMood = weatherMood;
}

// EYE TRACKING - Follow tilt IMMEDIATELY
if (!isShaking && !isAngry && currentPage == 0) {
    // Map tilt to pupil position (more sensitive)
    float rawX = currentRoll / 5.0;    // Max 18 pixels at 90 degrees
    float rawY = currentPitch / 5.0;

    targetPupilX = constrain(rawX, -12, 12);
    targetPupilY = constrain(rawY, -10, 10);

    // FAST smooth follow
    currentPupilX = currentPupilX * 0.6 + targetPupilX * 0.4;
    currentPupilY = currentPupilY * 0.6 + targetPupilY * 0.4;

    leftEye.targetPupilX = currentPupilX;
    leftEye.targetPupilY = currentPupilY;
    rightEye.targetPupilX = currentPupilX;
    rightEye.targetPupilY = currentPupilY;

    // Head follows tilt
    leftEye.targetX = 28 + (currentRoll / 15);
    leftEye.targetY = 18 + (currentPitch / 15);
    rightEye.targetX = 80 + (currentRoll / 15);
    rightEye.targetY = 18 + (currentPitch / 15);
}

// Store values
lastAccelX = accelX;
lastAccelY = accelY;
lastAccelZ = accelZ;

```

```

    prevRoll = currentRoll;
    prevPitch = currentPitch;
}

// =====
// WEATHER FUNCTIONS
// =====

const unsigned char* getBigIcon(String w) {
    if (w == "Clear") return bmp_clear;
    if (w == "Clouds") return bmp_clouds;
    if (w == "Rain" || w == "Drizzle") return bmp_rain;
    return bmp_clouds;
}

const unsigned char* getMiniIcon(String w) {
    if (w == "Clear") return mini_sun;
    if (w == "Rain" || w == "Drizzle" || w == "Thunderstorm") return
mini_rain;
    return mini_cloud;
}

void updateWeatherMood() {
    if (weatherMain == "Clear") weatherMood = MOOD_HAPPY;
    else if (weatherMain == "Rain" || weatherMain == "Drizzle")
weatherMood = MOOD_SAD;
    else if (weatherMain == "Thunderstorm") weatherMood =
MOOD_SURPRISED;
    else if (weatherMain == "Clouds") weatherMood = MOOD_NORMAL;
    else if (temperature > 35) weatherMood = MOOD_ANGRY;
    else if (temperature < 10) weatherMood = MOOD_SLEEPY;
    else weatherMood = MOOD_NORMAL;

    // Only update if not in special state
    if (!isShaking && !isAngry) {
        currentMood = weatherMood;
    }
}

void getWeatherAndForecast() {
    if (WiFi.status() != WL_CONNECTED) return;

    HTTPClient http;
    String url = "http://api.openweathermap.org/data/2.5/weather?q=" +
String(CITY) + "," + String(COUNTRY_CODE) + "&appid=" +
String(OPENWEATHER_API_KEY) + "&units=metric";
    http.begin(url);
    int httpCode = http.GET();
    if (httpCode == 200) {
        String payload = http.getString();
        JSONVar myObject = JSON.parse(payload);
        if (JSON.typeof(myObject) != "undefined") {
            temperature = double(myObject["main"]["temp"]);
            feelsLike = double(myObject["main"]["feels_like"]);
            humidity = int(myObject["main"]["humidity"]);
            weatherMain = (const char*)myObject["weather"][0]["main"];

```

```

        weatherDesc = (const char*)myObject["weather"][0]
["description"];
        if (weatherDesc.length() > 0) weatherDesc[0] =
toupper(weatherDesc[0]);
        updateWeatherMood();
    }
}

http.end();

url = "http://api.openweathermap.org/data/2.5/forecast?q=" +
String(CITY) + "," + String(COUNTRY_CODE) + "&appid=" +
String(OPENWEATHER_API_KEY) + "&units=metric";
http.begin(url);
httpCode = http.GET();
if (httpCode == 200) {
    String payload = http.getString();
    JSONVar fo = JSON.parse(payload);
    if (JSON.typeof(fo) != "undefined") {
        struct tm t;
        getLocalTime(&t);
        int today = t.tm_wday;
        const char* days[] = { "SUN", "MON", "TUE", "WED", "THU",
"FRI", "SAT" };
        int indices[3] = { 7, 15, 23 };
        for (int i = 0; i < 3; i++) {
            int idx = indices[i];
            fcast[i].temp = (int)double(fo["list"][idx]["main"]["temp"]);
            fcast[i].iconType = (const char*)fo["list"][idx]["weather"]
[0]["main"];
            int nextDayIndex = (today + i + 1) % 7;
            fcast[i].dayName = days[nextDayIndex];
        }
    }
}

http.end();
}

// =====
// TOUCH HANDLER
// =====
void handleTouch() {
    bool currentPinState = digitalRead(TOUCH_PIN);
    unsigned long now = millis();

    if (currentPinState && !lastPinState) {
        pressStartTime = now;
        isLongPressHandled = false;
    }
    else if (currentPinState && lastPinState) {
        if ((now - pressStartTime > LONG_PRESS_TIME) &&
!isLongPressHandled) {
            if (currentPage == 0 && !isShaking && !isAngry) {
                currentMood++;
                if (currentMood > MOOD_SUSPICIOUS) currentMood = 0;
                weatherMood = currentMood;
            }
        }
    }
}

```

```

    }
    else if (currentPage == 1) {
        subPage = (subPage == 1) ? 0 : 1;
    }
    else if (currentPage == 2) {
        subPage = (subPage == 2) ? 0 : 2;
    }
    isLongPressHandled = true;
}
}
else if (!currentPinState && lastPinState) {
    if ((now - pressStartTime < LONG_PRESS_TIME) &&
!isLongPressHandled) {
        tapCounter++;
        lastTapTime = now;
    }
}
lastPinState = currentPinState;

if (tapCounter > 0) {
    if (now - lastTapTime > DOUBLE_TAP_DELAY) {
        if (tapCounter == 1) {
            if (subPage != 0) {
                subPage = 0;
            } else {
                currentPage++;
                if (currentPage > 2) currentPage = 0;
            }
        }
        tapCounter = 0;
    }
}
}

// =====
// DRAW FUNCTIONS
// =====
void drawDizzyEye(Eye& e, bool isLeft) {
    int ix = (int)e.x;
    int iy = (int)e.y;
    int iw = (int)e.w;
    int ih = (int)e.h;
    int cx = ix + iw/2;
    int cy = iy + ih/2;

    // White background
    display.fillCircle(cx, cy, iw/2, SH110X_WHITE);

    // Animated circling pupils
    unsigned long now = millis();
    float angle = (now - shakeStartTime) * 0.025;
    int radius = iw / 3;

    int pupilX = cx + cos(angle) * radius;
    int pupilY = cy + sin(angle) * radius;

```

```

display.fillCircle(pupilX, pupilY, 4, SH110X_BLACK);
display.fillCircle(pupilX + 2, pupilY - 2, 1, SH110X_WHITE);

int pupilX2 = cx - cos(angle) * radius;
int pupilY2 = cy - sin(angle) * radius;
display.fillCircle(pupilX2, pupilY2, 3, SH110X_BLACK);

// Animated stars
int starOffset = (now / 80) % 4;
display.drawBitmap(6 - starOffset, 0, bmp_dizzy_stars, 16, 16,
SH110X_WHITE);
display.drawBitmap(106 + starOffset, 0, bmp_dizzy_stars, 16, 16,
SH110X_WHITE);
}

void drawAngryEye(Eye& e, bool isLeft) {
    int ix = (int)e.x;
    int iy = (int)e.y;
    int iw = (int)e.w;
    int ih = (int)e.h;

    // White sclera
    display.fillRoundRect(ix, iy, iw, ih, 8, SH110X_WHITE);

    // Angry pupil (smaller, shifted)
    int cx = ix + iw/2;
    int cy = iy + ih/2;
    int pupilSize = iw / 2.5;

    int px = cx - 2;
    int py = cy;

    display.fillRoundRect(px - pupilSize/2, py - pupilSize/2,
pupilSize, pupilSize, pupilSize/2, SH110X_BLACK);

    // Angry eyebrow
    if (isLeft) {
        for(int i = 0; i < 8; i++) {
            display.drawLine(ix - 2, iy - 2 + i, ix + iw + 2, iy + 6 + i,
SH110X_BLACK);
        }
        display.drawBitmap(ix - 12, iy - 6, bmp_angry_mark, 16, 16,
SH110X_WHITE);
    } else {
        for(int i = 0; i < 8; i++) {
            display.drawLine(ix - 2, iy + 6 + i, ix + iw + 2, iy - 2 + i,
SH110X_BLACK);
        }
        display.drawBitmap(ix + iw - 4, iy - 6, bmp_angry_mark, 16, 16,
SH110X_WHITE);
    }

    // Red veins (tiny lines)
    for(int i = 0; i < 3; i++) {

```

```

        display.drawLine(ix + 2 + i*4, iy + ih - 2, ix + 6 + i*4, iy + ih
- 6, SH110X_WHITE);
    }
}

void drawNormalEye(Eye& e, bool isLeft) {
    int ix = (int)e.x;
    int iy = (int)e.y;
    int iw = (int)e.w;
    int ih = (int)e.h;

    int r = 8;
    if (iw < 20) r = 3;
    display.fillRoundRect(ix, iy, iw, ih, r, SH110X_WHITE);

    int cx = ix + iw / 2;
    int cy = iy + ih / 2;

    int pw = iw / 2.2;
    int ph = ih / 2.2;

    int px = cx + (int)e.pupilX - (pw / 2);
    int py = cy + (int)e.pupilY - (ph / 2);

    if (px < ix) px = ix;
    if (px + pw > ix + iw) px = ix + iw - pw;
    if (py < iy) py = iy;
    if (py + ph > iy + ih) py = iy + ih - ph;

    display.fillRoundRect(px, py, pw, ph, r / 2, SH110X_BLACK);

    if (iw > 15 && ih > 15) {
        display.fillCircle(px + pw - 4, py + 4, 2, SH110X_WHITE);
    }

    // Expression masks
    if (currentMood == MOOD_HAPPY || currentMood == MOOD_LOVE) {
        display.fillRect(ix, iy + ih - 10, iw, 12, SH110X_BLACK);
    }
    else if (currentMood == MOOD_SLEEPY) {
        display.fillRect(ix, iy, iw, ih/2, SH110X_BLACK);
    }
    else if (currentMood == MOOD_SAD) {
        if (isLeft) {
            for(int i = 0; i < 8; i++) display.drawLine(ix, iy + i, ix +
iw, iy + 4 + i, SH110X_BLACK);
        } else {
            for(int i = 0; i < 8; i++) display.drawLine(ix, iy + 4 + i, ix
+ iw, iy + i, SH110X_BLACK);
        }
    }
}

void drawMouth() {
    int mx = 64;

```

```

int my = 54;

if (currentMood == MOOD_DIZZY && isShaking) {
    unsigned long now = millis();
    for(int i = -8; i <= 8; i++) {
        int y = my + sin(i * 0.8 + now * 0.03) * 4;
        display.drawPixel(mx + i, y, SH110X_WHITE);
    }
    return;
}

if (currentMood == MOOD_ANGRY && isAngry) {
    // Gritted teeth
    display.fillRect(mx - 8, my - 2, 16, 5, SH110X_BLACK);
    for(int i = -5; i <= 5; i+=3) {
        display.drawLine(mx + i, my - 2, mx + i, my + 2, SH110X_WHITE);
    }
    return;
}

switch(currentMood) {
    case MOOD_HAPPY:
        for(int i = -6; i <= 6; i++) {
            int y = my - (i*i / 16);
            if(y < my + 2) display.drawPixel(mx + i, y, SH110X_WHITE);
        }
        break;
    case MOOD_SAD:
        for(int i = -6; i <= 6; i++) {
            int y = my + 2 + (i*i / 16);
            display.drawPixel(mx + i, y, SH110X_WHITE);
        }
        break;
    case MOOD_SURPRISED:
        display.fillCircle(mx, my + 1, 3, SH110X_BLACK);
        display.drawCircle(mx, my + 1, 3, SH110X_WHITE);
        break;
    case MOOD_SLEEPY:
        display.drawLine(mx - 4, my, mx + 4, my, SH110X_WHITE);
        break;
    case MOOD_LOVE:
        display.drawBitmap(mx - 6, my - 2, bmp_heart, 12, 12,
SH110X_WHITE);
        break;
    default:
        display.drawLine(mx - 4, my, mx + 4, my, SH110X_WHITE);
        break;
}

}

void updateEyePhysics() {
    unsigned long now = millis();
    breathVal = sin(now / 800.0) * 1.5;

    int blinkDelay = (isShaking || isAngry) ? 60 : 120;

```

```

int blinkMin = (isShaking || isAngry) ? 300 : 2000;
int blinkMax = (isShaking || isAngry) ? 800 : 6000;

if (now > leftEye.nextBlinkTime) {
    leftEye.blinking = true;
    rightEye.blinking = true;
    leftEye.lastBlink = now;
    leftEye.nextBlinkTime = now + random(blinkMin, blinkMax);
}

if (leftEye.blinking) {
    leftEye.targetH = 2;
    rightEye.targetH = 2;
    if (now - leftEye.lastBlink > blinkDelay) {
        leftEye.blinking = false;
        rightEye.blinking = false;
        leftEye.targetH = 32;
        rightEye.targetH = 32;
    }
}

leftEye.update();
rightEye.update();
}

void drawEmoPage() {
    updateEyePhysics();

    if (currentMood == MOOD_DIZZY && isShaking) {
        drawDizzyEye(leftEye, true);
        drawDizzyEye(rightEye, false);
    }
    else if (currentMood == MOOD_ANGRY && isAngry) {
        drawAngryEye(leftEye, true);
        drawAngryEye(rightEye, false);
    }
    else {
        drawNormalEye(leftEye, true);
        drawNormalEye(rightEye, false);
    }

    drawMouth();

    // Floating particles
    if (!isShaking && !isAngry) {
        if (currentMood == MOOD_LOVE) {
            display.drawBitmap(56, 0, bmp_heart, 16, 16, SH110X_WHITE);
        } else if (currentMood == MOOD_SLEEPY) {
            display.drawBitmap(110, 0, bmp_zzz, 16, 16, SH110X_WHITE);
        }
    }
}

void drawClockPage() {
    struct tm t;

```

```

    if (!getLocalTime(&t)) {
        display.setFont(NULL);
        display.setCursor(30, 30);
        display.print("Syncing...");
        return;
    }

    String ampm = (t.tm_hour >= 12) ? "PM" : "AM";
    int h12 = t.tm_hour % 12;
    if (h12 == 0) h12 = 12;

    display.setTextColor(SH110X_WHITE);
    display.setFont(NULL);
    display.setCursor(114, 0);
    display.print(ampm);

    display.setFont(&FreeSansBold18pt7b);
    char timeStr[6];
    sprintf(timeStr, "%02d:%02d", h12, t.tm_min);
    int16_t x1, y1;
    uint16_t w, h;
    display.getTextBounds(timeStr, 0, 0, &x1, &y1, &w, &h);
    display.setCursor((SCREEN_WIDTH - w) / 2, 42);
    display.print(timeStr);

    display.setFont(&FreeSans9pt7b);
    char dateStr[20];
    strftime(dateStr, 20, "%a, %b %d", &t);
    display.getTextBounds(dateStr, 0, 0, &x1, &y1, &w, &h);
    display.setCursor((SCREEN_WIDTH - w) / 2, 60);
    display.print(dateStr);
}

void drawWorldClockPage() {
    struct tm t;
    if (!getLocalTime(&t)) return;

    time_t now;
    time(&now);

    time_t londonEpoch = now;
    time_t newyorkEpoch = now - (4 * 3600);

    struct tm* localtime = &t;
    struct tm* londontm = gmtime(&londonEpoch);
    struct tm* nytm = gmtime(&newyorkEpoch);

    display.fillRect(0, 0, 128, 14, SH110X_WHITE);
    display.setFont(NULL);
    display.setTextColor(SH110X_BLACK);
    String title = "WORLD CLOCK";
    int16_t x1, y1;
    uint16_t w, h;
    display.getTextBounds(title, 0, 0, &x1, &y1, &w, &h);
    display.setCursor((SCREEN_WIDTH - w) / 2, 4);

```

```

display.print(title);
display.setTextColor(SH110X_WHITE);

display.drawLine(42, 16, 42, 63, SH110X_WHITE);
display.drawLine(85, 16, 85, 63, SH110X_WHITE);

display.setFont(NULL);
display.setTextSize(0);

display.setCursor(12, 18);
display.print("JPR");
display.setCursor(52, 18);
display.print("LDN");
display.setCursor(94, 18);
display.print("NYC");

display.setFont(NULL);
display.setTextSize(1);

char jStr[10];
sprintf(jStr, "%02d:%02d", localtime->tm_hour, localtime->tm_min);
display.getTextBounds(jStr, 0, 0, &x1, &y1, &w, &h);
display.setCursor(21 - (w / 2), 48);
display.print(jStr);

char lStr[10];
sprintf(lStr, "%02d:%02d", localtime->tm_hour, localtime->tm_min);
display.getTextBounds(lStr, 0, 0, &x1, &y1, &w, &h);
display.setCursor(64 - (w / 2), 48);
display.print(lStr);

char nStr[10];
sprintf(nStr, "%02d:%02d", localtime->tm_hour, localtime->tm_min);
display.getTextBounds(nStr, 0, 0, &x1, &y1, &w, &h);
display.setCursor(106 - (w / 2), 48);
display.print(nStr);
}

void drawWeatherPage() {
    if (WiFi.status() != WL_CONNECTED) {
        display.setFont(NULL);
        display.setCursor(0, 0);
        display.print("No WiFi");
        return;
    }

    display.drawBitmap(96, 0, getBigIcon(weatherMain), 32, 32,
SH110X_WHITE);
    display.setFont(&FreeSansBold9pt7b);
    String c = CITY;
    c.toUpperCase();
    display.setCursor(0, 14);
    if (c.length() > 9) c = c.substring(0, 8) + ".";
    display.print(c);
}

```

```

display.setFont(&FreeSansBold18pt7b);
int tempInt = (int)temperature;
display.setCursor(0, 48);
display.print(tempInt);

int16_t x1, y1;
uint16_t w, h;
display.getTextBounds(String(tempInt).c_str(), 0, 48, &x1, &y1, &w,
&h);
display.fillCircle(x1 + w + 5, 26, 4, SH110X_WHITE);

display.setFont(NULL);
display.drawBitmap(88, 32, bmp_tiny_drop, 8, 8, SH110X_WHITE);
display.setCursor(100, 32);
display.print(humidity);
display.print("%");
display.setCursor(88, 45);
display.print("~");
display.print((int)feelsLike);
display.drawLine(0, 52, 128, 52, SH110X_WHITE);
display.setCursor(0, 55);
String shortDesc = weatherDesc;
if (shortDesc.length() > 15) shortDesc = shortDesc.substring(0, 13)
+ "...";
display.print(shortDesc);
}

void drawForecastPage() {
display.fillRect(0, 0, 128, 14, SH110X_WHITE);
display.setFont(NULL);
display.setTextColor(SH110X_BLACK);
String title = "3-DAY FORECAST";
int16_t x1, y1;
uint16_t w, h;
display.getTextBounds(title, 0, 0, &x1, &y1, &w, &h);
display.setCursor((SCREEN_WIDTH - w) / 2, 4);
display.print(title);
display.setTextColor(SH110X_WHITE);

display.drawLine(42, 16, 42, 63, SH110X_WHITE);
display.drawLine(85, 16, 85, 63, SH110X_WHITE);

for (int i = 0; i < 3; i++) {
int xStart = i * 43;
int centerX = xStart + 21;

display.setFont(NULL);
String d = fcast[i].dayName;
if (d == "") d = "WAIT";
display.setCursor(centerX - (d.length() * 3), 18);
display.print(d);

display.drawBitmap(centerX - 8, 26,
getMiniIcon(fcast[i].iconType), 16, 16, SH110X_WHITE);

```

```

        display.setFont(&FreeSansBold9pt7b);
        int16_t x1, y1;
        uint16_t w, h;
        display.getTextBounds(String(fcast[i].temp).c_str(), 0, 0, &x1,
&y1, &w, &h);
        display.setCursor(centerX - (w / 2) - 2, 58);
        display.print(fcast[i].temp);
        display.fillCircle(centerX + (w / 2) + 1, 50, 2, SH110X_WHITE);
    }
}

void playBootAnimation() {
    display.setTextColor(SH110X_WHITE);
    int cx = 64;
    int cy = 32;

    for (int r = 0; r < 80; r += 4) {
        display.clearDisplay();
        display.fillCircle(cx, cy, r, SH110X_WHITE);
        display.display();
        delay(8);
    }

    for (int r = 0; r < 80; r += 4) {
        display.clearDisplay();
        display.fillCircle(cx, cy, 80, SH110X_WHITE);
        display.fillCircle(cx, cy, r, SH110X_BLACK);
        display.display();
        delay(8);
    }

    display.setFont(&FreeSansBold9pt7b);
    String bootText = "N I M O";
    int16_t x1, y1;
    uint16_t w, h;
    display.getTextBounds(bootText, 0, 0, &x1, &y1, &w, &h);
    display.clearDisplay();
    display.setCursor((SCREEN_WIDTH - w) / 2, 36);
    display.print(bootText);
    display.display();
    delay(2000);
}

// =====
// SETUP
// =====
void setup() {
    Serial.begin(115200);
    delay(500);

    Wire.begin(SDA_PIN, SCL_PIN);
    pinMode(TOUCH_PIN, INPUT_PULLUP);

    display.begin(0x3C, true);
    display.setTextColor(SH110X_WHITE);

```

```

display.clearDisplay();

// Initialize MPU6050
mpu6050.begin();
calibrateMPU6050();

// Initialize eyes
leftEye.init(28, 18, 32, 32);
rightEye.init(80, 18, 32, 32);

playBootAnimation();

// Connect to WiFi
display.clearDisplay();
display.setFont(NULL);
display.setCursor(20, 30);
display.print("Connecting WiFi");
display.display();

WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
unsigned long wifiStart = millis();
while (WiFi.status() != WL_CONNECTED && (millis() - wifiStart <
15000)) {
    delay(500);
    display.print(".");
    display.display();
}

if (WiFi.status() == WL_CONNECTED) {
    display.clearDisplay();
    display.setCursor(25, 30);
    display.print("WiFi Connected!");
    display.display();
    delay(1000);

    configTime(0, 0, "pool.ntp.org");
    setenv("TZ", TIMEZONE, 1);
    tzset();
    getWeatherAndForecast();
    lastWeatherUpdate = millis();
} else {
    display.clearDisplay();
    display.setCursor(15, 30);
    display.print("WiFi Failed!");
    display.display();
    delay(2000);
}
}

// =====
// LOOP
// =====
void loop() {
    handleTouch();
    updateMotion();

```

```

    if (millis() - lastWeatherUpdate > 600000 && WiFi.status() ==
WL_CONNECTED) {
        getWeatherAndForecast();
        lastWeatherUpdate = millis();
    }

    display.clearDisplay();

    if (currentPage == 0) {
        drawEmoPage();
    }
    else if (currentPage == 1) {
        if (subPage == 1) drawWorldClockPage();
        else drawClockPage();
    }
    else if (currentPage == 2) {
        if (subPage == 2) drawForecastPage();
        else drawWeatherPage();
    }

    display.display();
    delay(20);
}

```

Code Features:

✓ Physics-Based Eye Movement: Smooth acceleration/deceleration for realistic eye tracking

✓ MPU6050 Motion Detection: Real-time tilt tracking and shake detection

✓ Automatic Mood Transitions: Shake → Dizzy → Angry automatically

✓ Touch Navigation: Tap to change screens, long press to change moods

✓ WiFi Weather: Live weather data and 3-day forecast

✓ NTP Time Sync: Accurate time with multiple time zones

✓ Blinking Animation: Natural random blinking intervals

Setup Instructions:

✓ Change WIFI_SSID and WIFI_PASSWORD to your WiFi credentials

✓ Get a free API key from OpenWeatherMap and replace OPENWEATHER_API_KEY

✓ Set your CITY and COUNTRY_CODE (e.g., "London", "UK")

✓ Set your TIMEZONE (e.g., "IST-5:30" for India, "EST-5" for New York)

✓ Install required libraries: Adafruit_GFX, Adafruit_SH110X, MPU6050_tockn, Arduino_JSON

✓ Select "Xiao ESP32-C3" board and upload!

Technical Specifications

✓ Microcontroller: Seeed Studio Xiao ESP32-C3 (RISC-V, 160MHz)

✓ Display: 1.3" OLED SH1106 (128x64 pixels)

✓ Motion Sensor: MPU6050 (6-axis gyro + accelerometer)

✓ Touch Sensor: TTP223 Capacitive Touch Module

✓ Battery: 3.7V LiPo 1000mAh with TP4056 charger

✓ Power Consumption: ~100mA (screen on, WiFi connected)

✓ Battery Life: 8-10 hours per charge

✓ WiFi: 2.4GHz 802.11 b/g/n

✓ Emotions: 10 different moods with unique eye/mouth animations

Applications



Fun animated buddy for your desk



Educational

Learn about sensors, displays, and animations



Robotics

Base for expressive robot projects



Gift Project

Unique handmade gift for tech lovers

Design Your Own PCBs with Altium

For designing professional PCBs like custom versions of NIMO, I use Altium — a platform that makes electronics design easier, faster, and more connected.

For students, they have **Altium Student Lab**. Just sign up with your university email for free access to step-by-step PCB courses and industry-recognized certifications.

 [Start building your future with Altium](#)

 ***Note:** Keep NIMO still during the first 5 seconds for calibration! After that, tilt it to see the eyes follow your movement. Shake it to trigger the dizzy → angry reaction. Tap the touch sensor to navigate between screens!*



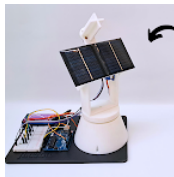


Enter Comment

Popular posts from this blog

Solar Tracking System

- December 01, 2024



Dual-Axis Solar Tracking System Using Arduino Build a Dual-Axis Solar Tracking System Using Arduino In this project, we'll create a DIY dual-axis solar tracking system that adjusts a solar panel's or ...

[READ MORE](#)

Arduino Code

- February 16, 2024

```
//define Pins #include <Servo.h> Servo servo; int trigPin = 11; int echoPin = 12; //
defines variables long duration; int distance; void setup() { servo.attach(13);
servo.write(180); delay(2000); // Sets the trigPin as an Output pinMode(' ...
```

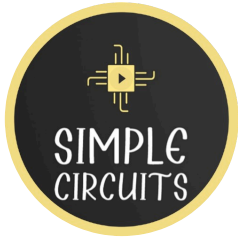
[READ MORE](#)

Arduino Code Car Parking System

- March 07, 2024

```
// Created by Simple Circuits #include <Wire.h> #include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27,16,2); #include <Servo.h> Servo myservo; int IR1 = 2; int
IR2 = 3; int Slot = 4; //Total number of parking Slots int flag1 = 0; int f ...
```

[READ MORE](#)



MR. SIMPLE

[VISIT PROFILE](#)

Archive



[Report Abuse](#)